

SIGFOX

0G Technology. LPWAN dedicated to Massive IoT.



Introduction

Sigfox is a company founded in 2009 that builds wireless networks to connect low-energy objects such as electricity meters, smartwatches, and washing machines, which need to be continuously on and emitting small amounts of data. Sigfox employs a proprietary technology that enables communication using the Industrial, Scientific and Medical ISM radio band, which uses 868 MHz in Europe and 902 MHz in the US. It utilizes a wide-reaching signal that passes freely through solid objects, called "ultra-narrowband" and requires little energy, being termed "Low-power Wide-area network (LPWAN)". The network is based on a one-hop star topology and requires a mobile operator to carry the generated traffic. The signal can also be used to easily cover large areas and to reach underground objects.

Sigfox has partnered with a number of firms in the LPWAN industry, such as Texas Instruments and Silicon Labs. The ISM radio band supports bidirectional communication. The existing standard for Sigfox communications supports up to **140 uplink messages a day**, each of which can carry a payload of **12 Bytes** (Excluding message header and transmission information), and up to 4 downlink messages per day, each of which can carry a payload of 8 Bytes. For more details about Sigfox, please visit the [Sigfox Developer Portal](#) ↗.

This documentation will describe how to integrate SigFox devices and their data into the Thinger.io Platform. In the first steps, we will review how to configure Thinger.io resources, and then, on the Sigfox side, we will configure the communication with the platform for pushing our sensor's data.

Integrating a Sigfox Device with Thinger.io

This process is carried out in two parts: on the one hand, the preparation of Thinger.io to receive data from Sigfox and, on the other hand, the configuration of the Sigfox cloud callback that will send the information to Thinger.io. During the next sections, we will explain both parts, starting with Thinger.io side steps:

There are two ways to configure Thinger.io to work with Sigfox devices. The best option is by deploying the "Sigfox Plugin", which will manage the integration, providing advanced features such as device auto-provisioning (good to integrate large networks), Uplink/Downlink payload processing and device management, but this option is only available for subscribed developers. Freemium accounts can also make individual Sigfox device integration using the "HTTP device". Both ways are explained below:

Advanced Integration (with Sigfox plugin)

> **SigFox Plugin**

Single Device Integration (without plugins)

When implementing little prototypes or maker projects using the free account, it is possible to integrate an individual device using the "HTTP device" that allows using almost every Thinger.io platform feature, including:

- Store data in buckets
- Show data in customizable dashboards
- Send endpoints to post data on emails, social networks or third parties

- Sigfox downlink processes to send configuration data to the device

 Payload data processing is only available using plugin integration

To perform this integration, it is required to create a new HTTP device and configure its callback flows as it is explained in the HTTP devices section of this documentation:

HTTP DEVICES



Once the new device has been created, Thinger.io will provide a REST API callback that can be used to configure the Sigfox cloud, as it is explained in the section below:

Sigfox Cloud Configuration

After making all the configurations that are required to get Thinger.io ready for receiving data, the next step is to configure the Sigfox Backend for pushing data to it, using our token identifier and the token we have generated.

Creating Sigfox Callback

In this step, we will create a Sigfox callback that will push the information from our Sigfox device to our Thinger.io data bucket. In our example, a callback is just an endpoint that is called when the Sigfox device sends data over the network, so we will configure the callback to point to our data bucket.

To create a callback in Sigfox:

1. Go to <https://backend.sigfox.com> and log in to the account. It is assumed that the device has already been registered with the platform.
2. Click on `Device Type` tab on the top, and then click on the desired device type name to configure. Alternatively, navigate to the `Device` tab and click on the `Device type` column of the device.
3. Click on `Callbacks` on left menu, and then create a new one.

In this step, select the option to create a `Custom Callback`, as there is a need to call an endpoint not directly supported by the Sigfox back-end.

The screenshot displays the Sigfox web interface for creating a new callback. The left sidebar lists navigation options: INFORMATION, LOCATION, ASSOCIATED DEVICES, DEVICES BEING TRANSFERRED, STATISTICS, EVENT CONFIGURATION, and CALLBACKS. The main content area is titled "Device type 'SmartEverything' - New Callback". It includes a description of callbacks and a "Custom callback" section with a cloud icon. Below this, there are three other options: AWS IoT, AWS Kinesis, and Microsoft Azure Event hub. The footer shows a URL and copyright information.

Device type 'SmartEverything' - New Callback

Create callbacks to connect Sigfox cloud to your server/platform.
A callback is a custom http request containing your device(s) data, along with other variables, sent to a given server/platform when the aforesaid device(s) message is received by Sigfox cloud.

Custom callback
Creates a new callback from Sigfox cloud to your own server. This is the "default" callback type. You can create a full custom request (http method, content type, headers, etc).

AWS IoT
AWS IoT is a managed cloud platform that lets connected devices easily and securely interact with cloud applications and other devices. AWS IoT can support billions of devices and trillions of messages, and can process and route those messages to AWS endpoints and to other devices reliably and securely.

AWS Kinesis
Amazon Kinesis is a platform for streaming data on AWS, offering powerful services to make it easy to load and analyze streaming data, and also providing the ability for you to build custom streaming data applications for specialized needs.

Microsoft Azure™ Event hub

<https://backend.sigfox.com/devicetype/58f72ac99e93e17e4ef23d0f/callbacks/new> Copyright © Sigfox - 6.4.6 - 257 - Terms and conditions / Cookie policy.

Then, we need to configure the callback to write to our data bucket. Here is the configuration. Details for each field are provided after this:

Type DATA UPLINK

Channel URL

Send duplicate ☐

Custom payload config temp::float:32:little-endian hum::float:32:little-endian ?

URL syntax: http://host/path?id={device}&time={time}&key1={var1}&key2={var2}...
 Available variables: device, time, duplicate, snr, station, data, avgSnr, lat, lng, rssi, seqNumber
 Custom variables: customData#temp, customData#hum

Url pattern https://api.thinger.io/v1/users/alvarolb/buckets/SmartEverything/data

Use HTTP Method POST

Send SNI ☐ (Server Name Indication) for SSL/TLS connections

Headers Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJkbWFydEV2Z2 ✕

header	value

Content type application/json

Body

```
{
  "device": "{device}",
  "snr": {snr},
  "rssi": {rssi},
  "station": "{station}",
  "latitude": {lat},
  "longitude": {lng},
  "temperature": {customData#temp},
  "humidity": {customData#hum}
}
```

The configuration in our example is:

1. Type is DATA with UPLINK, as we want to send our device data.
2. Channel is of type URL, as we will be calling an HTTP endpoint.
3. Send duplicate as disabled to avoid writing duplicate messages received by different base stations.
4. Custom payload config will completely depend on the payload sent by the device. In our case, our device will be sending the temperature and humidity as 32-bit floats, so we have configured the payload as temp::float:32:little-endian hum::float:32:little-endian, where we define the temp and hum parameters as 32-bit floats in little-endian. Notice that Sigfox only supports 12 bytes of payload per message, so it is a must to optimize this space, like sending temperature and humidity as integers if it is not required decimal accuracy. For example, this will work.

5. **Url pattern** must be configured according to the Thingier.io user ID and our bucket name.
- The pattern should be like
`https://api.thingier.io/v1/users/{user_id}/buckets/{bucket_id}/data`.
 - The `{user_id}` and `{bucket_id}` must be changed to match the account. For example, the final URL pattern will be
`https://api.thingier.io/v1/users/alvarolb/buckets/SmartEverything/data`.
- Note that Sigfox variables can also be used to compose the URL; for instance, to store data from each device in a different bucket, a URL could be created:
`https://api.thingier.io/v1/users/alvarolb/buckets/{device}/data`.
6. **HTTP Method** should be set to POST.
7. In **Headers** we must include an **Authorization** header with our device token in order to authenticate the bucket write request.
- Header name should be **Authorization**
 - Header value should be **Bearer {access_token}**, where the `{access_token}` token is generated in the previous steps.
 - This is the example final header value. Note the space between **Bearer** and the token itself:
- ```
Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiJUbWYdEV2ZXJ5dGhpbmciLCJ1c3IiOiJhbHZhcm9sYiJ9.0Qb48c_ToBiIVcC0dvXU2Kn51mTnGLDcN44shVRzNls
```
8. The final step is to configure the **Body** and its **Content type**. For content type, we will set **application/json** as the bucket can store arbitrary JSON data. The body will then contain all the information we want to store, formatted in JSON. In Sigfox, the body can be defined using available variables, which include those provided by the platform (such as device ID, link quality, or device location) and those defined by the payload configuration. In our case, we defined variables `temp`, and `hum`, that will be included with other Sigfox variables. For this example, the payload is:

```
{
 "device" : "{device}",
 "snr" : {snr},
 "rssi" : {rssi},
 "station": "{station}",
 "latitude": {lat},
 "longitude": {lng},
 "temperature" : {customData#temp},
 "humidity" : {customData#hum}
}
```

Notice that we are mixing Sigfox variables, like `{device}`, with our own custom data in the payload, like `{customData#temp}`. This body is then processed on every message reception, and the variables will be replaced with the current values. So, the server will receive a JSON payload with the device identifier, device temperature, humidity, coarse location (km accuracy), and signal quality.

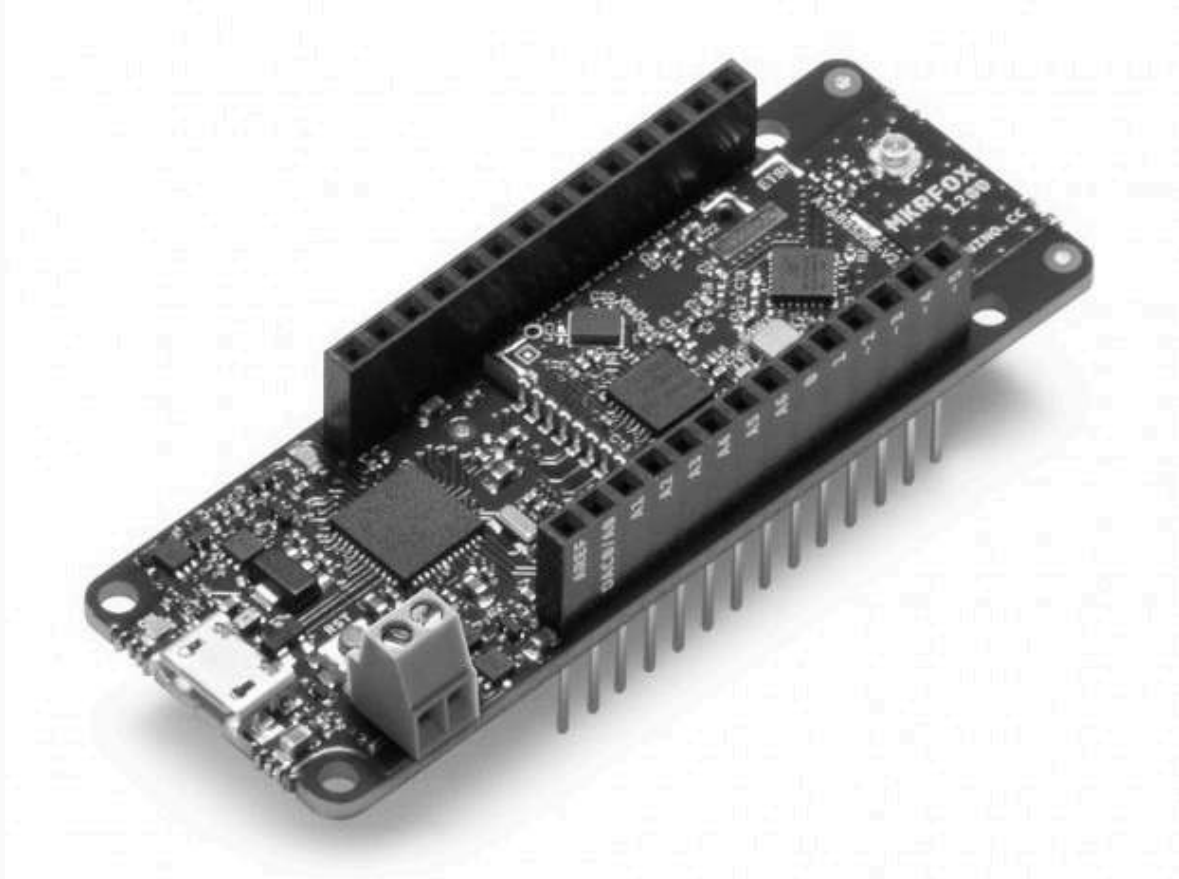
After these steps, we should now have a callback completely configured to push data to our data bucket.

## Programming Sigfox Devices

Now it is time to program our Sigfox Device that will be sending data to our buckets. In this case, we provide examples for the [SmartEverything](#) ⚡ device and the [Arduino MKRFOX1200](#) ⚡.

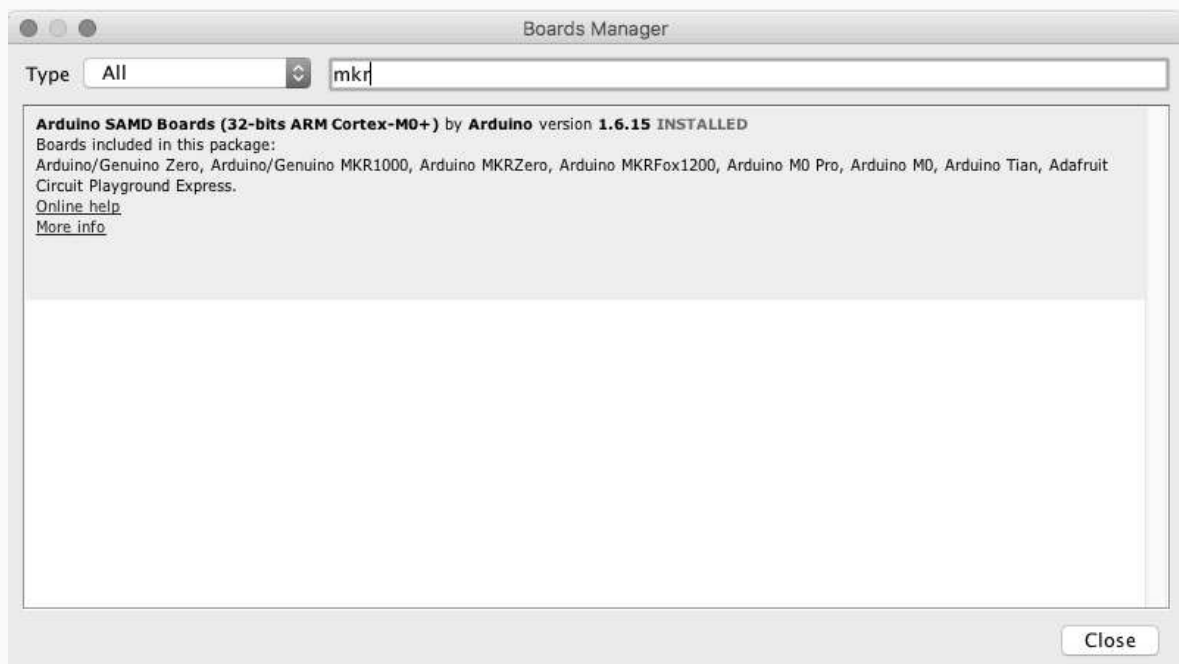
### Arduino MKRFOX1200

Arduino MKRFOX1200 has been designed to offer a practical and cost-effective solution for makers seeking to add SigFox connectivity to their projects with minimal previous experience in networking. It is based on the Microchip SAMD21 and an ATA8520 SigFox module. Can run for over six months on 2 AA 1.5V batteries with typical usage. The design includes the ability to power the board using two 1.5V AA or AAA batteries or an external 5V.



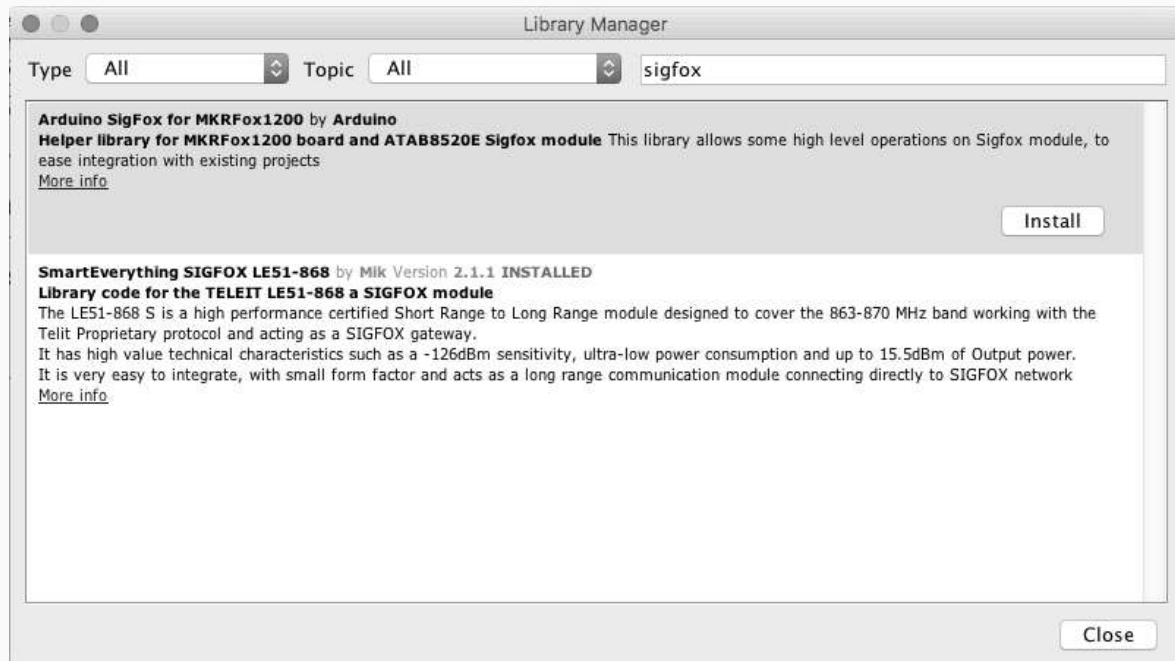
## Initial Setup

To program this device, we will use the [Arduino IDE](#) ٨. In this case, it is necessary to install or update the board toolchain, which can be done directly from the Boards Manager, searching for `mrk`, and selecting the Arduino SAMD Boards.





Install the `Arduino SigFox for MKRFox1200` library that is available from the Library Manager, and it is also **NECESSARY** to install the `Arduino Low Power`, and the `RTCZero` libraries.



After a successful installation, we can now select the Board in the Arduino IDE. Just select the Arduino MKRFOX12000. Select, as with any other Arduino board, the port where the device is connected.



Check that everything is up and running by flashing this example, which will provide information about the module, like the board ID and PAC. This information is necessary for registering the device in Sigfox.

```

#include <SigFox.h>

void setup() {
 Serial.begin(9600);

 while(!Serial) {};

 if (!SigFox.begin()) {
 Serial.println("Shield error or not present!");
 return;
 }

 String version = SigFox.SigVersion();
 String ID = SigFox.ID();
 String PAC = SigFox.PAC();

 // Display module information
 Serial.println("MKRFox1200 Sigfox first configuration");
 Serial.println("SigFox FW version " + version);
 Serial.println("ID = " + ID);
 Serial.println("PAC = " + PAC);

 Serial.println("");

 Serial.print("Module temperature: ");
 Serial.println(SigFox.internalTemperature());

 Serial.println("Register your board on
https://backend.sigfox.com/activate with provided ID and PAC");

 delay(100);

 // Send the module to the deepest sleep
 SigFox.end();
}

void loop() {
 // put your main code here, to run repeatedly:
}

```

**Notice:** From this point on, it is assumed that the board has already been registered on the Sigfox account. If not, refer to the [First Configuration](#) ↗ tutorial from Arduino.

### Pushing data to Sigfox

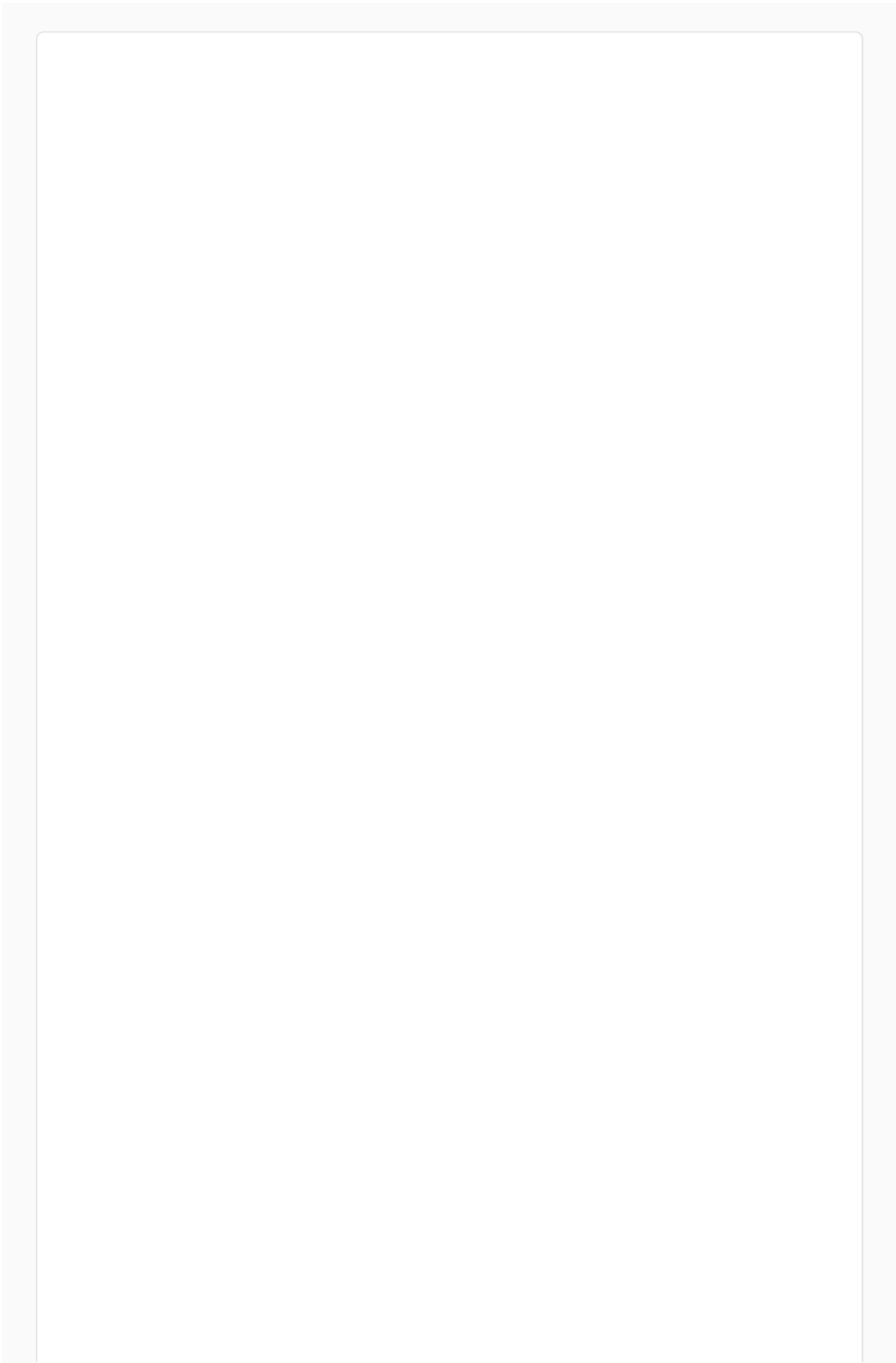
Now that we have our toolchain running, it is time to code something to push data to the Sigfox Backend. Before presenting the code, **remember** that in the callback we have defined in the Sigfox, we established a payload config that is expecting to receive two floats representing both temperature and humidity. So, our payload must match this definition:

```
temp::float:32:little-endian hum::float:32:little-endian
```

In our code, this payload can be easily represented by a `struct` that holds two floats. Defining custom structs with different data types is possible, but **structure padding** and **architecture** must be carefully considered. The **Sigfox payload** will require reconfiguration to ensure proper decoding of the transmitted fields.

```
struct data{
 float temp;
 float hum;
};
```

In this case, we are using the Arduino MKRFOX1200 along with a DHT sensor providing temperature and humidity required for the callback we have configured in the Sigfox back-end. If a DHT sensor is unavailable, the board's internal temperature sensor can be utilized by calling `SigFox.internalTemperature()`, and setting the humidity value to zero or any other value.



```

#include <SigFox.h>
#include <SimpleDHT.h>
#include <ArduinoLowPower.h>

#define DHT11_PIN 0

void setup() {
 Serial.begin(9600);
 pinMode(LED_BUILTIN, OUTPUT);
}

void blink(unsigned int count, unsigned long ms){
 for(int i=0; i<count; i++){
 digitalWrite(LED_BUILTIN, HIGH);
 delay(ms);
 digitalWrite(LED_BUILTIN, LOW);
 delay(ms);
 }
}

void send_data(){
 //Initialize Sigfox module
 SigFox.begin();
 delay(100);

 // Enable debug LED and disable automatic deep sleep
 SigFox.debug();

 // clears all pending interrupts
 SigFox.status();
 delay(1);

 // define Sigfox payload data structure
 struct data{
 float temp;
 float hum;
 };

 // read temperature and humidity from DHT sensor connected at
 pin DHT11_PIN
 SimpleDHT11 dht11;
 byte temp, hum;
 dht11.read(DHT11_PIN, &temp, &hum, NULL);

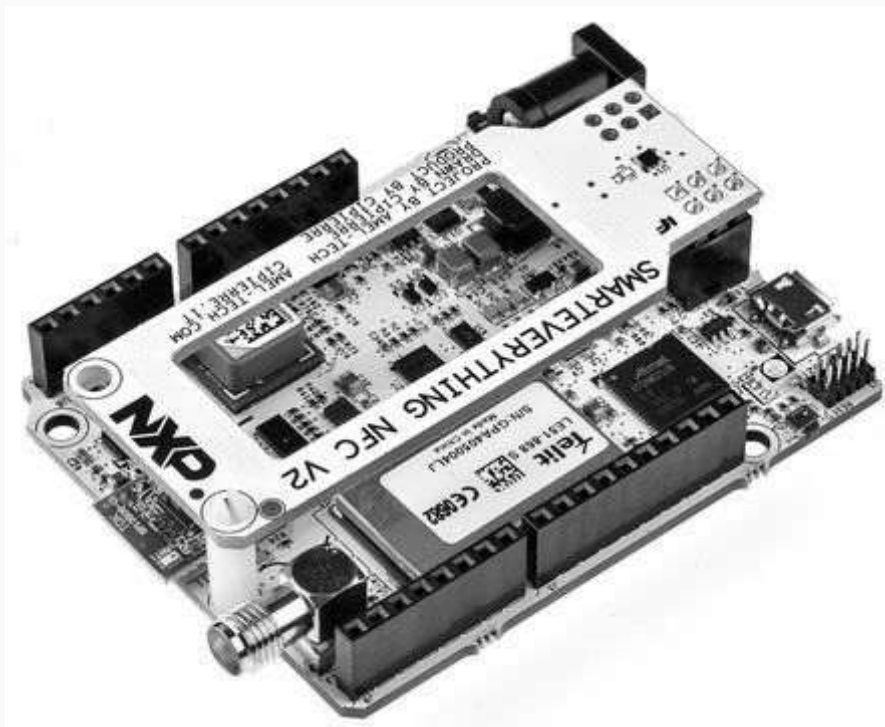
 // NOTE! It is not quite efficient sending bytes as floats over
 the net, but this is just for illustrative purposes
 struct data reading;
 reading.temp = temp;

```

**Notice,** The `LowPower.sleep` function call can be uncommented, and the standard `sleep` function call commented out, to enable deep sleep on the Arduino MKRFOX1200, which is beneficial when operating on batteries. It is possible to avoid using the `Serial`, and the `SigFox.debug()` that is, they're just for debugging purposes. In sleep mode, the device requires a manual reset before flashing it again.

## SmartEverything

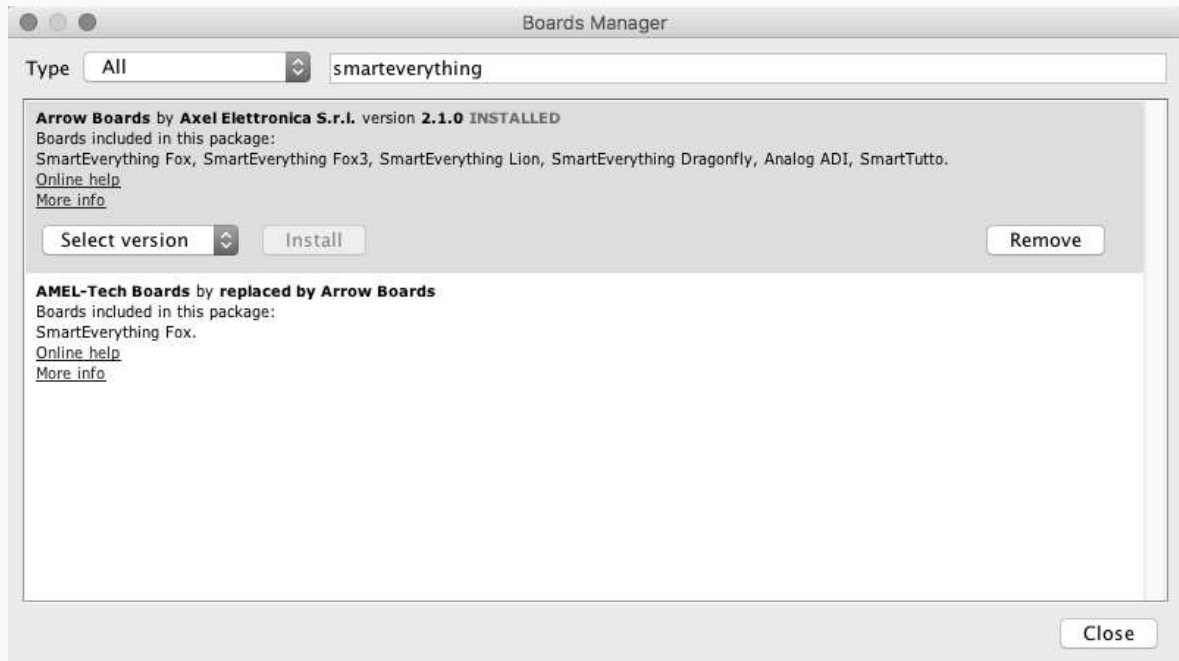
SmartEverything is an IoT device specially designed for rapid prototyping, as it has full Arduino compatibility, with multiple sensors ready to use, like MEMS Pressure Sensor, Proximity and Ambient Light Sensor, iNEMO 9-axis inertial module, humidity and temperature sensors, and even NFC NTAG, or a GPS/GNSS integrated antenna. If these features are quite interesting by themselves, this board also integrates a Bluetooth Low Energy (BLE) and, of course, a Sigfox Module (Telit LE51-868 S 868MHz module).



With these awesome features, we can use the board for multiple purposes, like vehicle tracking with the GPS, building a micro meteorological station, registering vibrations and impacts with the accelerometers, or any other use case. For this example, we will register just the temperature and humidity. This way, we have created a simple code that will register temperature and humidity every 10 minutes.

### Initial Setup

To program this device, we will use the [Arduino IDE](#) 7. In this case, it is necessary to install the board toolchain, which can be done directly from the Boards Manager, searching for `smarteverything` and selecting the Arrow Boards by Axel Elettronica.



After a successful installation, we can now select the Board in the Arduino IDE. Just select the SmartEverything Fox (Native USB Port). Select, as with any other Arduino board, the port where the device is connected.

## SmartEverything Boards

SmartEverything Fox (via Atmel-ICE)

✓ SmartEverything Fox (Native USB Port)

SmartEverything Fox3 (via Atmel-ICE)

SmartEverything Fox3 (Native USB Port)

SmartEverything Lion (via Atmel-ICE)

SmartEverything Lion (Native USB Port)

SmartEverything Dragonfly (via Atmel-ICE)

SmartEverything Dragonfly (Native USB Port)

Analog ADI (via Atmel-ICE)

Analog ADI (Native USB Port)

SmartTutto (via Atmel-ICE)

SmartTutto (Native USB Port)

### Pushing data to Sigfox

Now it is time to write a simple sketch to send our sensor readings to Sigfox. The provided sample sketch will basically initialize, in the setup, the Sigfox Modem, the sensors, and the USB Serial port for some debugging. Then, in the loop, our sketch will read both the temperature and humidity and will transmit the data to Sigfox. It will also check if the transmission is OK to blink a green LED on success or a red LED otherwise. After that, it will sleep for 10 minutes, as we mentioned in the introduction, Sigfox will allow only 140 messages a day.

Before presenting the code, **remember** that in the callback we have defined in the Sigfox, we established a payload config that is expecting to receive two floats representing both temperature and humidity. So, our payload must match this definition:

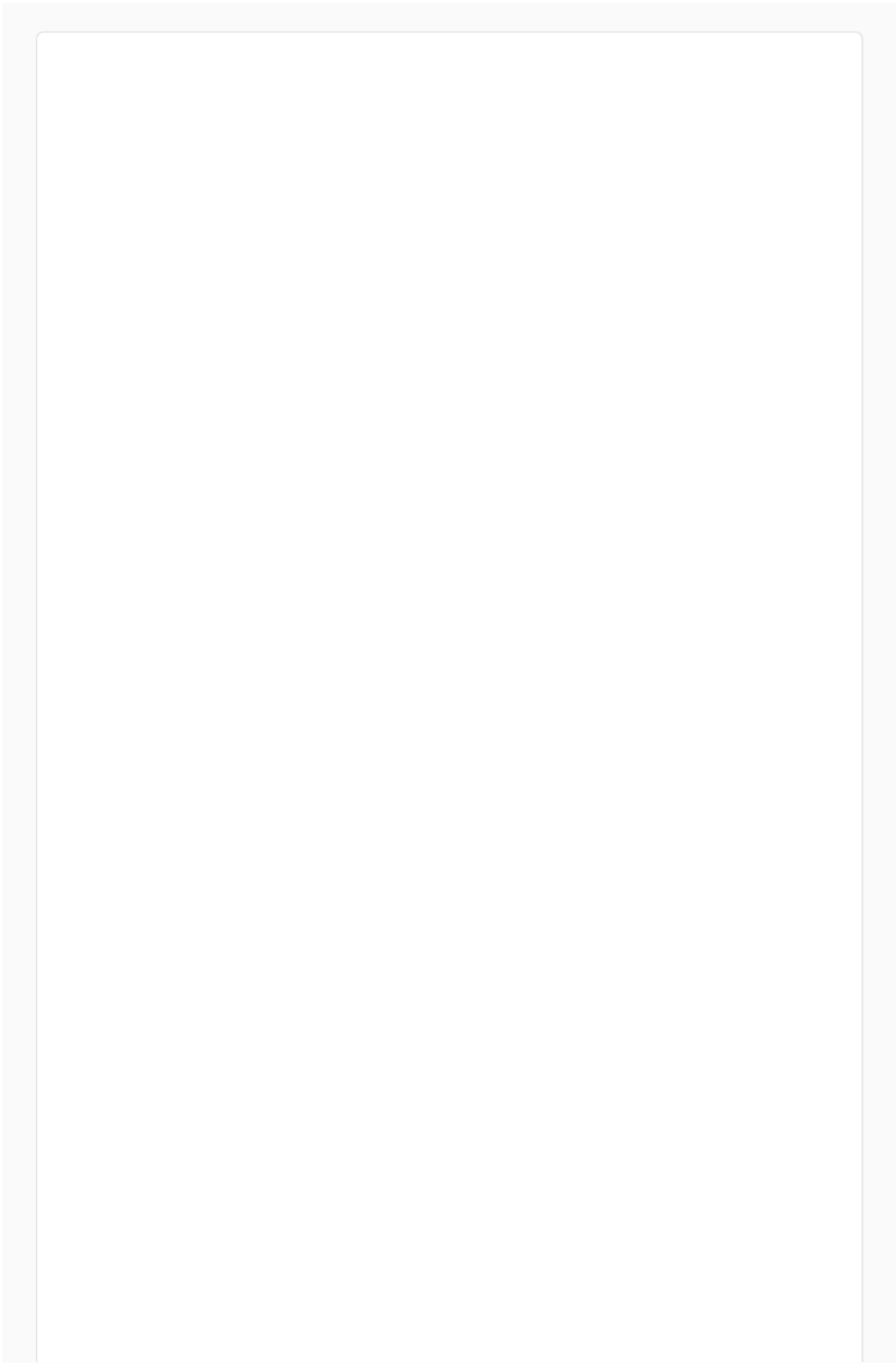
```
temp::float:32:little-endian hum::float:32:little-endian
```



In the code, this payload can be easily represented by a `struct` containing two floats. It is possible to define custom structs with different data types (though **structure padding** and **architecture** should be considered). However, the **Sigfox payload** must be reconfigured to properly decode the fields being sent.

```
struct data{
 float temp;
 float hum;
};
```

**Notice** that this code has not been optimized for battery-powered use cases. Use the power-saving mode on the device if needed, but this is out of the scope of this example.



```

#include <Wire.h>
#include <SmeSFX.h>
#include <Arduino.h>
#include <HTS221.h>

void setup() {
 // init temp & hum sensor
 Wire.begin();
 smeHumidity.begin();

 // init serial
 SerialUSB.begin(115200);

 // init sigfox module
 sfxAntenna.begin(19200, &SigFox);
 sfxAntenna.setSfxDataMode();
}

void send_data(){
 // define Sigfox payload data structure
 struct data{
 float temp;
 float hum;
 };

 // read sensor data into the struct
 struct data reading;
 reading.temp = smeHumidity.readHumidity();
 reading.hum = smeHumidity.readTemperature();

 // send the structure to Sigfox (8 bytes)
 SerialUSB.println("Sending SigFox message!");
 sfxAntenna.sfxSendData((const char*)&reading, sizeof(reading));
}

void loop() {
 // send Sigfox data
 send_data();

 // wait for a response
 bool response=false;
 do{
 if (sfxAntenna.hasSfxAnswer()) {
 switch (sfxAntenna.sfxDataAcknoledge()) {
 case SFX_DATA_ACK_OK:
 ledGreenLight(HIGH);
 SerialUSB.println("Answer OK! :)");
 delay(2000);

```

## Checking Sigfox Setup

After we have both the device code running, the Sigfox callback configured, and the data bucket created, we should check that everything is up and running.

We can start by checking that the Sigfox platform is receiving our messages. Just go to the device in the Sigfox back-end, and open the `Messages` section that is on the left panel. Here, some messages have been received. See the payload being sent (in hexadecimal), and some other information like link quality, timestamp, or callback result.

| Time                | Data / Decoding  | Location                                                                            | Link quality                                                                          | Callbacks                                                                             |
|---------------------|------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| 2017-04-22 22:22:58 | 5a7dcf41e1182342 |    |    |    |
| 2017-04-22 22:12:50 | e0f1cf4130272242 |   |   |   |
| 2017-04-22 22:02:42 | 4e94ce4110462442 |  |  |  |
| 2017-04-22 21:52:35 | f9c3bb41aec44c42 |  |  |  |
| 2017-04-22 21:36:59 | a264cb4154c12542 |  |  |  |
| 2017-04-22 21:26:52 | 45c9ca4137612642 |  |  |  |

It is interesting here to check that our callback response is successful, as the callback icon changes from green to red depending on the result. In our case, our callbacks are in green, so the request was ok. Click on the icon to see the server response, which is a 200 OK HTTP response.

### Callback - OK



[OK] - Base station 06C9 - < 1 second

200 - OK

[POST] <https://api.thinger.io/v1/users/alvarolb/buckets/SmartEverything/data>

Then we can also check that our data bucket is being populated with the data received from Sigfox. So, open the data bucket in Thinger.io. Nice! We have our data now being stored. **Notice** that the columns in the bucket are just the fields we configured in the Sigfox callback body.

#### Bucket Explorer

| Date                         | device | humidity  | latitude | longitude | rsi  | snr   | station | temperature |
|------------------------------|--------|-----------|----------|-----------|------|-------|---------|-------------|
| 2017-04-23T01:03:30.688+0200 | C378F  | 41.874535 | 41       | -4        | -130 | 21.28 | 06C9    | 25.32931    |
| 2017-04-23T00:53:22.895+0200 | C378F  | 41.72929  | 41       | -4        | -132 | 19.29 | 06C9    | 25.386208   |
| 2017-04-23T00:43:14.984+0200 | C378F  | 41.384327 | 41       | -4        | -129 | 21.93 | 06C9    | 25.518965   |
| 2017-04-23T00:33:07.300+0200 | C378F  | 41.03574  | 41       | -4        | -130 | 21.36 | 06C9    | 25.765518   |
| 2017-04-23T00:22:59.397+0200 | C378F  | 40.774296 | 41       | -4        | -130 | 21.01 | 06C9    | 25.936207   |
| 2017-04-23T00:12:51.492+0200 | C378F  | 40.53827  | 41       | -4        | -129 | 21.62 | 06C9    | 25.993103   |
| 2017-04-23T00:02:43.686+0200 | C378F  | 41.06842  | 41       | -4        | -129 | 22.02 | 06C9    | 25.822414   |
| 2017-04-22T23:52:35.800+0200 | C378F  | 51.19207  | 41       | -4        | -130 | 20.77 | 1D62    | 23.47069    |
| 2017-04-22T23:37:00.903+0200 | C378F  | 41.438797 | 41       | -4        | -131 | 19.81 | 1D62    | 25.424137   |
| 2017-04-22T23:26:53.076+0200 | C378F  | 41.594936 | 41       | -4        | -133 | 18.21 | 06C9    | 25.348276   |

Refresh

Viewing 0 to 54 items

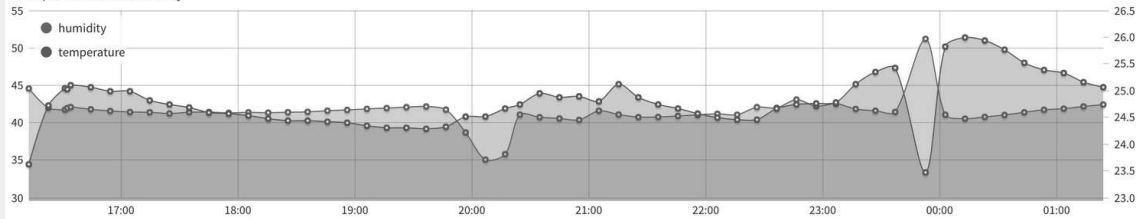
## Building a Dashboard

Now that we have our data in the bucket, we can just create a real-time dashboard from our Sigfox data. Create the widgets by selecting the bucket as the data source, and that's all!

## Sigfox Dashboard



### Temperature & Humidity



### Sigfox device location



### Link Quality



### Monitored Device

C378F

### Sigfox Station

06C9

Previous  
REMOTE CONSOLE

Next  
LoRaWAN

Last updated 5 months ago

Was this helpful?

